

CMCMIS · PHASE 8

DASHBOARD + INQUIRY MODULE WITH SUB-MODULES

End-to-End Technical Documentation · SEALED · LOCKED · DS

Field	Value
Document name	phase8fullySEALEDsealpacked.docx
Project	CMCMIS_SIMPLIFIED · SAC / ISRO-style Calibration MIS
Phase	Phase 8 — Dashboard + Inquiry Module with Sub-Modules
Status	END-TO-END COMPLETED · SEALED · LOCKED · GREEN
Prepared for	Deep Sorathiya (DS)
Documentation mode	Senior Technical Documentation Engineer · no-code architectural record
Implementation style	JavaScript-only stack · React/Vite/Tailwind · Node/Express · MySQL final
Core doctrine	Live DB only · no mock data · repo aliasing · ADD-only migrations · permission-aware UI

SEAL This document records what Phase 8 achieved, why the implementation was designed this way, and how a senior engineer should think through database, backend, frontend, permission, cache, and verification logic. It intentionally avoids heavy source listings; only small code-shaped fragments are used to explain doctrine.

Generated from the Phase 8 completion transcript and locked CMCMIS project doctrine. Internal render-verified before delivery.

Table of Contents

Part	Section	Purpose
Part I	Mission & sealed context	Why Phase 8 exists and what it inherits
Part II	Database reality and schema doctrine	Live table mapping, surprises, canonical aliases
Part III	Decision register P8-D1 → P8-D15	Locked choices that shape every file
Part IV	Build walkthrough	Step 0 → Step 13, end-to-end
Part V	Dashboard module deep dive	ORG/MY variants, KPI service, cache, UI
Part VI	Inquiry module deep dive	Four tabs, FULLTEXT, permissions, pagination
Part VII	Backend logic development	Layering, validators, repo, service, controller, routes
Part VIII	Frontend logic development	API wrappers, hooks, pages, URL state
Part IX	Security, RBAC, performance	Permission gates, no SQL injection, latency
Part X	Verification and acceptance	A1 → A16 final green transcript
Part XI	File inventory and sealed manifest	DB, BE, FE, docs, patches, deletes
Part XII	Slice 2 punchlist	Pre-approved next scope

Part I — Mission & Sealed Context

MISSION Phase 8 is the visibility layer of CMC MIS. Earlier phases created identity, equipment, requests, cards, and admin control. Phase 8 turns that operational data into fast role-aware dashboard KPIs and a global inquiry/search experience.

Phase 8 was not a redesign phase. It was a disciplined continuation of the Phase 4 → Phase 7 doctrine: use the existing MySQL final database, map legacy column names in the repository layer, keep controllers thin, keep services business-aware, keep migrations additive, and make the UI permission-aware.

The module has two major surfaces: the Dashboard module and the Inquiry module. The Dashboard provides role-aware KPI cards and quick actions. The Inquiry module provides four search sub-modules: Vendors, Products, Job Cards, and Instruments.

Inherited phase	What it contributes to Phase 8	Why it matters
Phase 0 → 3	Discovery, roles, requirements, legacy DB audit, migration runner, permission matrix	Phase 8 never guesses table names; it introspects and maps real legacy tables.
Phase 4	Auth/RBAC HTTP layer, JWT, authenticate, authorize, ProtectedRoute, sidebar permission logic	Every Phase 8 endpoint is guarded; every FE route is permission-aware.
Phase 5	Equipment module pattern and cmms_equip_mst aliasing doctrine	Dashboard KPIs depend on equipment status and calibration due dates.
Phase 6	Job Request + Job Card foundation, status fields, audit pairing, row-level thinking	Dashboard and Inquiry read from JR/JC workflow state.
Phase 7	Admin Users + Employees, tokenVersionCache, LRU pattern, SESSION_REVOKED, URL and route discipline	Phase 8 reuses the in-process LRU pattern for kpiCache.

HOW TO THINK Phase 8 is not about beautiful cards first. It is about asking: Which live table owns each number? Which role is allowed to see it? Which index makes it fast? Which frontend state keeps it stable?

Phase 8 One-Line Version

Dashboard answers: "What needs my attention right now?" Inquiry answers: "Can I find any operational object quickly without opening multiple modules?"

Phase 8 Scope Boundary

Included in Slice 1	Deferred to Slice 2 / later
Role-aware /dashboard/kpis endpoint	Charts: workload bar, calibration timeline, status pie
ORG and MY dashboard variants	Recent activity stream from audit_log
Quick action buttons gated by permissions	SSE push for instant KPI refresh
Four Inquiry tabs: Vendors, Products, Job Cards, Instruments	Inquiry row click → deep detail page links
FULLTEXT + prefix search with pagination	Service-provider vendor enum extension
A1 → A16 smoke-tested acceptance	Division-calibration KPI for LE/LIC

Part II — Database Reality & Schema Doctrine

The first serious engineering move in Phase 8 was not writing a React component. It was Step 0 read-only introspection against the live database. This prevented the system from being built against imaginary table names such as vendors, products, equipment, job_requests, and job_cards when the actual runtime tables are legacy cmms_* tables.

PRINCIPLE Repo aliasing remains the master doctrine: every service/controller/frontend layer speaks canonical names, but the repository layer alone knows real legacy column names such as EQM_CAL_DUE_DATE and JR_MVP_STATUS.

Prompt / canonical idea	Real DB table	Approx rows	Phase 8 usage
vendors	cmms_cont_mst	540	Vendor Inquiry tab; type aliasing and contact search
products	cmms_product_mst	32	Product Inquiry tab; thin product master with derived equipment count
equipment	cmms equip_mst	5,705	Instrument Inquiry tab and Dashboard calibration/utilization KPIs
job_requests	cmms_jobrequest_mst	21,490	Dashboard pending/active/completed request KPIs
job_cards	cmms_jobcard_mst	19,432	Dashboard completed week and Job Card Inquiry tab
divisions	cmms_section_mst	293	Division labels and location display
audit log	audit_log	42 at introspection time	Future activity feed; already present

Step 0 Surprises That Changed the Design

Surprise	Reality	Engineering decision
No OPERATIONAL equipment status	The real status enum uses ACTIVE as the healthy state.	UI may say Operational, but SQL filters on EQM_MVP_STATUS = ACTIVE.
No SERVICE_PROVIDER vendor enum	Vendor types are MFR, VENDOR, BOTH, OEM.	Slice 1 dropdown ships Manufacturer / Supplier / All only.
Product table is thin	cmms_product_mst has no manufacturer/category/supplier FK.	Product tab shows derived Equipment Count and Top Manufacturer instead of inventing columns.
Personal fields use employee IDs	JR_SUBMITTEDBYID and EQM_CREATED_BY are varchar(7), not user_id.	Cache key and personal KPIs use req.user.employeeId.

Canonical Field Map — Dashboard-Load Bearing Columns

Feature	Canonical meaning	Real table.column	Index / note
Pending Jobs	submitted requests waiting for action	cmms_jobrequest_mst.JR_MVP_STATUS = SUBMITTED	covered by JR status indexes
Calibration Due	active equipment due soon	cmms equip_mst.EQM_CAL_DUE_DATE + EQM_MVP_STATUS	new idx equip_status_caldue
Completed This Week	verified/finished job cards	COALESCE(JM_VERIFIED_ON, JM_JobEndDate)	new idx_jc_status_verified + idx_jc_status_ended
Equipment Utilization	active equipment with open job work	cmms equip_mst + cmms_jobcard_mst	computed via EXISTS / distinct work logic
My Active Requests	user-owned active requests	JR_SUBMITTEDBYID = req.user.employeeId	uses employee_id
My Calibration Due	equipment created by me due soon	EQM_CREATED_BY = req.user.employeeId	new idx equip_creator_caldue

LOCKED REALITY No new business tables were created for Phase 8. The phase is index-only + permission-safety-net + read module code. This preserves legacy production data and respects the sealed database doctrine.

Part III — Phase 8 Decision Register P8-D1 → P8-D15

ID	Decision	Locked choice	Reason
P8-D1	Dashboard route	One route /dashboard; backend branches by role.	One URL, one mental model, server owns visibility logic.
P8-D2	KPI cache	In-process LRU, 5000 entries, 10 second TTL.	No Redis in stack; Phase 7 already proved LRU pattern.
P8-D3	Cache key namespace	kpi:org and kpi:personal:emp:<employee_id>.	Personal scope is employee_id because DB uses varchar(7) IDs.
P8-D4	Personal calibration scope	EQM_CREATED_BY = req.user.employeeId.	Closest existing auditable field; no ALTER.
P8-D5	Completed timestamp	COALESCE(JM_VERIFIED_ON, JM_JobEndDate).	Works for verified and completed rows.
P8-D6	Operational equipment	UI label Operational = DB status ACTIVE.	Legacy enum has no OPERATIONAL.
P8-D7	Vendor type aliasing	MFR/OEM/BOTH → Manufacturer; VENDOR/BOTH → Supplier.	No SERVICE_PROVIDER in DB enum.
P8-D8	Product tab	ID, name, description, equipment count, top manufacturer derived.	Avoids fake manufacturer/category/supplier columns.
P8-D9	KPI refresh	FE 30 sec poll; staleTime 25 sec; BE TTL 10 sec.	Fresh enough without hammering DB.
P8-D10	Week boundary	ISO week Monday 00:00.	Managers think in calendar weeks.
P8-D11	Inquiry search	<3 chars prefix LIKE; >=3 chars FULLTEXT BOOLEAN-prefix.	Works for oscill → oscilloscope/oscillator.
P8-D12	Pagination	page_size = 10 or 25, default 10.	Inquiry is compact and fast.
P8-D13	Row click	No-op hover in Slice 1.	Detail routes deferred; no broken links.
P8-D14	Tab state	URL only: /inquiry?tab=vendors products job-cards instruments.	Shareable, reload-safe, no localStorage.
P8-D15	Cache invalidation	JR/equipment mutations invalidate org + relevant personal key.	Write-time freshness with TTL safety net.

DECISION STYLE Every decision is tied to actual database reality. We did not create design wishes; we converted schema facts into maintainable implementation choices.

Part IV — End-to-End Build Walkthrough

Order	Action	Engineering purpose
Step 0	Read-only DB introspection	Proved the real tables and columns before coding.
Step 1	SCHEMA_PHASE8.md	Canonical-to-real mapping, decisions register, gap list.
Step 2	Q-1 → Q-9 sign-off	DS accepted recommended answers and locked deviations.
Step 3	Migration 120	4 KPI BTREE indexes for dashboard aggregation.
Step 4	Migration 121	3 FULLTEXT indexes for Inquiry tabs.
Step 5	Migration 122	Permission safety-net for Dashboard + Inquiry.
Step 6	kpiCache utility	LRU 10s TTL, 5000 entries, org/personal cache keys.
Step 7	Dashboard backend module	validators → repo → service → controller → routes.
Step 8	Inquiry backend module	four search endpoints, zod, repo aliasing, status maps.
Step 9	server.js wiring	Mounted /api/v1/dashboard and /api/v1/inquiry routes.
Step 10	Cache invalidation hooks	jobRequests + equipment services invalidate KPI cache after commit.
Step 11	Frontend API/hook/schemas	dashboard.js, inquiry.js, useDashboardKpis, useInquirySearch, inquirySchemas.
Step 12	Dashboard UI	DashboardHeader, KpiGrid, KpiCard, QuickActions.
Step 13	Inquiry UI	Inquiry.jsx plus tab/results/search components.
Step 14	Routing/nav	App.jsx + permission navigation; old placeholders removed.
Step 15	Smoke A1 → A16	All green; latency under targets.

The Build Order Logic

HOW TO THINK The order is database → backend contract → frontend surface → verification. Dashboard and Inquiry are read-heavy modules; if indexes and alias maps are wrong, every beautiful UI component becomes a slow or incorrect view.

```
DB introspection → SCHEMA_PHASE8.md → migrations
↓
Backend repo aliases → services → controllers → routes
↓
Frontend API wrappers → hooks → pages → route/nav
↓
A1..A16 smoke → sealed docs
```

Step 0 — Read-only Database Introspection

WHAT WE DID The phase began by probing actual table existence, columns, indexes, row counts, and status enums. This caught the absence of canonical vendors/products/equipment/job_requests/job_cards tables and confirmed the real cmms_* legacy tables. It also surfaced the three design-changing facts: no OPERATIONAL enum, no SERVICE_PROVIDER enum, and a thin product table.

Step 1 — SCHEMA_PHASE8.md

WHAT WE DID The schema document became the binding map for repository implementation. It documents canonical fields, real columns, index requirements, permissions, invariants, and open questions. Every repo SELECT uses aliasing from this document.

Step 2 — DS Sign-off

WHAT WE DID The Q-1 through Q-9 recommendations and three newly surfaced sub-questions were accepted with a green flag. This

prevented mid-build redesign and allowed implementation to proceed under locked assumptions.

Step 3 — ADD-only Database Migrations

WHAT WE DID Migrations 120, 121, and 122 were written as idempotent, additive scripts. They add indexes and permission safety-net rows only. No table was dropped, renamed, or structurally modified.

Step 4 — Backend Layer Construction

WHAT WE DID Dashboard and Inquiry followed the established five-file module template: validators, repo, service, controller, routes. This keeps SQL isolated, business logic testable, HTTP thin, and route wiring auditable.

Step 5 — Frontend Replacement of Placeholders

WHAT WE DID The old Dashboard and Inquiry placeholders were replaced by full module pages. The frontend uses thin API wrappers and hooks rather than directly embedding axios calls in visual components.

Step 6 — Verification

WHAT WE DID A1 through A16 checked role variants, cache hit behavior, permission denial, search branches, pagination, empty states, and p50 latency. All were green at seal time.

Part V — Dashboard Module Deep Dive

The Dashboard module is role-aware. Normal User and View-Only users receive the MY variant. Lab Engineer, Lab In-Charge, and Super Admin receive the ORG variant. The frontend does not decide this; it renders whatever `/dashboard/kpis` returns.

Layer	File / concept	Responsibility
Validator	dashboard.validators.js	Accepts no query params in Slice 1; strict empty schema.
Repo	dashboard.repo.js	Read-only aggregation SQL for ORG and MY KPI groups.
Service	dashboard.service.js	Role variant decision, cache read/write, card shaping, quick-action payloads.
Controller	dashboard.controller.js	Thin req.user → service → JSON response bridge.
Routes	dashboard.routes.js	GET <code>/api/v1/dashboard/kpis</code> guarded by authenticate + authorize dashboard:view.
Utility	kpiCache.js	In-process LRU cache shared by reader and invalidators.

ORG Dashboard Variant

Card	Value logic	Subtitle logic	Deep link
Pending Jobs	COUNT JR where JR_MVP_STATUS = SUBMITTED	+N today from last 24h	<code>/job-requests?status=SUBMITTED</code>
Calibration Due	COUNT active equipment due within 7 days	Within 7 days	<code>/equipment?calibration_due=7d</code>
Completed This Week	COUNT completed/verified job cards this ISO week	% delta vs last week	<code>/job-cards?status=COMPLETED&period=this_week</code>
Equipment Utilization	open-work equipment / active equipment × 100	Optimal / below / above range	<code>/equipment</code>

MY Dashboard Variant

Card	Value logic	Subtitle logic	Deep link
Active Requests	JR_SUBMITTEDBYID = employeeId and active statuses	N pending approval	<code>/job-requests?mine=1</code>
In Progress	JR_SUBMITTEDBYID = employeeId and IN_PROGRESS	Being processed / nothing in progress	<code>/job-requests?mine=1&status=IN_PROGRESS</code>
Completed This Month	owner + VERIFIED_CLOSED + current month	% delta vs last month	<code>/job-requests?mine=1&status=VERIFIED_CLOSED&period=this_month</code>
Due for Calibration	EQM_CREATED_BY = employeeId and due within 30 days	Next 30 days	<code>/equipment?registered_by_me=1&calibration_due=30d</code>

KPI Cache Doctrine

PATTERN Cache is read-side optimization, not a correctness boundary. Writes still go to MySQL. Cache invalidation happens after successful transaction commit. TTL is the fallback if a hook is missed.

kpi:org → shared by LE / LIC / SA
kpi:personal:emp:<employee_id> → one key per Normal / View-Only
TTL: 10 seconds · capacity: 5000 entries · FE poll: 30 seconds

Cache scenario	Expected behavior	Why it matters
First dashboard call	Cache miss, repo aggregation executes	Fresh data generated.
Second call inside TTL	cacheHit=true and cacheAgeMs > 0	DB protected from repeated aggregation.
Job Request submit	ORG and personal cache invalidated after commit	Pending/request counts refresh quickly.

Cache scenario	Expected behavior	Why it matters
Equipment create	ORG and creator personal cache invalidated after commit	Calibration and equipment counts refresh.

Part VI — Inquiry Module Deep Dive

Inquiry is a global lookup surface. It is intentionally read-only in Slice 1 and split into four sub-modules: Vendors, Products, Job Cards, and Instruments. The route state is URL-driven via tab query parameters, allowing reloads and shared links to preserve the selected tab.

Sub-module	Endpoint	Permission	Source table	Search method
Vendors	GET /inquiry/vendors	inquiry:search-vendors	cmms_cont_mst	type filter + FULLTEXT / prefix LIKE
Products	GET /inquiry/products	inquiry:search-products	cmms_product_mst + derived equipment joins	FULLTEXT / prefix LIKE
Job Cards	GET /inquiry/job-cards	inquiry:search-job-cards	cmms_jobcard_mst + cmms_eqip_mst	job-code prefix + equipment FULLTEXT
Instruments	GET /inquiry/instruments	inquiry:search-instruments	cmms_eqip_mst + cmms_section_mst	FULLTEXT / ID prefix

Search Branching Logic

DECISION Natural Language FULLTEXT was replaced by BOOLEAN prefix mode because user searches like oscill must match oscilloscope and oscillator. This was the one material code-level change surfaced by smoke testing.

Input length	Branch	Reason
0 chars	default page sorted by stable key	Allows tab load without search.
1–2 chars	prefix LIKE on safe primary/display fields	Avoids MySQL FULLTEXT minimum token pitfalls.
3+ chars	FULLTEXT BOOLEAN MODE with term* wildcards	Supports partial word lookup while staying indexed.

Vendor Inquiry

Column displayed	Source	Notes
Vendor Code	synthetic V-### from CMM_CONT_ID	Display-friendly only.
Name	CMM_CONT_NAME	Searchable.
Type	mapped from CMM_CONT_TYPE	Manufacturer or Supplier.
Contact	mobile / phone fallback	Safe display.
Email	CMM_CONT_EMAIL	Searchable.
Address	city + state	Lightweight row context.

Product Inquiry

Column displayed	Source	Reason
Product Code	synthetic P-### from PROD_ID	Stable UI code.
Product Name	PROD_NAME	Searchable.
Description	PROD_DESC	Searchable.
Equipment Count	derived from cmms_eqip_mst.EQM_INST_TYPE	Gives product operational weight.
Top Manufacturer	derived from equipment manufacturer joins	Better than fake supplier/category columns.

Job Card Inquiry

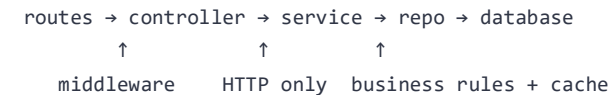
Column displayed	Source	Notes
Job Code	JM_SectionJobNo	Primary legacy job identifier.
Equipment	join to cmms_eqip_mst	May be null if legacy link incomplete.
Status	JM_MVP_STATUS	Mapped to user-friendly label.
Assigned Engineer	JM_AttendedBy / closest legacy field	Read-only.
Received / Completed / Verified Dates	JM_JCRecdDate, JM_JobEndDate, JM_VERIFIED_ON	Timeline context.

Instrument Inquiry

Column displayed	Source	Notes
Equipment Code	EQ-<type>-<id> synthetic	Composite PK converted to user-facing code.
Name	EQM_NAME	Searchable.
Model / Serial	EQM_MODELNO / EQM_SRNO	Searchable.
Division / Location	cmms_section_mst join	Context for lab ownership.
Status	EQM_MVP_STATUS	ACTIVE displayed as operational.
Last / Next Calibration	derived last verified JC + EQM_CAL_DUE_DATE	Calibration timeline.

Part VII — Backend Logic Development Doctrine

LAYERING LAW Phase 8 keeps the same import direction as earlier modules: validators define inputs, repo owns SQL, service owns decisions, controller owns HTTP glue, routes own middleware order.



Validators

Validators are not just field checks; they are contract firewalls. Dashboard uses a strict empty query object in Slice 1. Inquiry uses bounded q strings, page constraints, and canonical vendor type values only.

Module	Validation logic	Purpose
Dashboard	strict empty query schema	Prevents silent unknown filters before Slice 2.
Vendors	q max 100, type in MANUFACTURER/SUPPLIER, page_size 10/25	Stops unsupported SERVICE_PROVIDER.
Products	q max 100, page_size 10/25	Keeps thin product search predictable.
Job Cards	q max 100, page_size 10/25	Keeps global job-card lookup bounded.
Instruments	q max 100, page_size 10/25	Protects FULLTEXT search from massive strings.

Repositories

The repository layer translates canonical service intent into real SQL. It is the only layer allowed to know CMM_CONT_TYPE, EQM_MVP_STATUS, JR_MVP_STATUS, and other legacy names. This is why Phase 8 can read a legacy DB without contaminating the rest of the app.

PITFALL AVOIDED If a React component or controller compares status to OPERATIONAL, the system is wrong. The repo/service map ACTIVE to operational meaning centrally.

Services

Services convert raw query counts into business payloads: card labels, subtitles, percent delta strings, variant selection, quick actions, utilization bands, progress mappings, and permission-safe tab behavior.

Service responsibility	Dashboard	Inquiry
Role branching	variantForRole(role) → my/org	permission comes from route; tab payload is standardized
Business mapping	ACTIVE → Operational, delta labels, utilization bands	legacy type/status maps to UI labels
Cache	reads and writes kpiCache	no cache in Slice 1; searches are fast indexed reads
Output contract	cards + quick_actions + metadata	items + pagination + searchMeta

Controllers and Routes

Controllers remain thin. Routes mount authenticate → authorize → validate → controller. That makes route security inspectable by reading a single file and keeps route permissions aligned with the RBAC matrix.

Endpoint	Middleware chain	Response
/api/v1/dashboard/kpis	authenticate → authorize(dashboard:view) → validate(empty) → kpis	role-aware KPI payload
/api/v1/inquiry/vendors	authenticate → authorize(inquiry:search-vendors) → validate(vendorsQuery) → vendors	vendor results
/api/v1/inquiry/products	authenticate → authorize(inquiry:search-products) → validate(productsQuery) → products	product results
/api/v1/inquiry/job-cards	authenticate → authorize(inquiry:search-job-cards) → validate(jobCardsQuery) → jobCards	job card results
/api/v1/inquiry/instruments	authenticate → authorize(inquiry:search-instruments) → validate(instrumentsQuery) → instruments	instrument results

Part VIII — Frontend Logic Development Doctrine

The frontend was built as a thin consumer of backend contracts. It does not decide whether a user is ORG or MY; it renders the cards returned by the server. It does not hard-code search result data shapes in multiple places; it uses tab components and shared hooks.

Frontend File Families

Family	Files / components	Responsibility
API wrappers	lib/api/dashboard.js, lib/api/inquiry.js	Thin axios wrappers, return unwrapped data.
Schemas	lib/schemas/inquirySchemas.js	Tab and query parameter validation / helper constants.
Hooks	useDashboardKpis, useInquirySearch	Polling, debounce, AbortController, stale-safe state.
Dashboard page	pages/dashboard/*	Header, KPI grid, quick actions, refresh.
Inquiry page	pages/inquiry/*	URL tab state, search box, tab results, pagination.
Routing	App.jsx	ProtectedRoute + Layout around the new pages.
Navigation	permissions.js	Shows /dashboard and /inquiry based on granted permissions.

Dashboard UI Thinking

UI RULE Dashboard cards are not independent components with hidden business logic. They are dumb renderers of backend-defined cards. This is what prevents role drift between UI and backend.

UI component	What it renders	Logic it should NOT own
DashboardHeader	Title, last updated, manual refresh	KPI computation.
KpiGrid	Four cards in responsive grid	Role branching.
KpiCard	label, value, subtitle, icon, accent, link	SQL filters or status mapping.
QuickActions	permission-gated action buttons	Server-side permission truth.

Inquiry UI Thinking

URL DOCTRINE The selected tab lives in the URL, not component memory and not localStorage. /inquiry?tab=products must open Products directly after reload or share.

Concern	Frontend mechanism	Why
Tab state	URL search param tab=...	Shareable, refresh-safe, testable.
Search input	debounced local input → hook query	Avoids a request per keystroke.
Cancellation	AbortController in hook	Avoids stale result flashes.
Pagination	page + page_size from response	Backend remains source of truth.
No-match state	friendly empty-state component	Smoke A15 confirmed.

Part IX — Security, RBAC, Performance & Correctness

RBAC Matrix

Permission	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
dashboard:view	Yes	Yes	Yes	Yes	Yes
inquiry:search-vendors	Yes	Yes	Yes	Yes	Yes
inquiry:search-products	Yes	Yes	Yes	Yes	Yes
inquiry:search-instruments	Yes	Yes	Yes	Yes	Yes
inquiry:search-job-cards	No	Yes	Yes	Yes	Yes

PERMISSION TRUTH Normal User cannot access Job Card Inquiry; A9 confirms a 403 FORBIDDEN. That is not a UI-only hide. The backend route denies it.

Performance Strategy

Hot path	Optimization	Smoke result
/dashboard/kpis cache-hit	kpiCache LRU returns payload without DB aggregation	3–4 ms
/dashboard/kpis cache-miss	4 KPI query groups with new indexes	3 ms in smoke environment
/inquiry/vendors q=tech	FULLTEXT ft_cont_search	6 ms
/inquiry/instruments q=oscill	FULLTEXT BOOLEAN-prefix ft_equip_search	30 ms
Short q=ab	prefix LIKE branch	2 hits, <50 ms

SQL Safety

SECURITY Every search term is parameter-bound with mysql2 placeholders. Column names are allow-listed by endpoint and never interpolated from the user. Validators bound q length and page_size.

Database Migration Safety

Migration	Action	Safety property
120__phase8_kpi_indexes.sql	Adds 4 composite BTREE indexes	Idempotent guard via information_schema; no DDL destruction.
121__phase8_fulltext_indexes.sql	Adds 3 FULLTEXT indexes	Idempotent guard; uses existing InnoDB tables.
122__phase8_dashboard_inquiry_permissions.sql	INSERT IGNORE for 5 permissions and grants	Safety-net only; no duplicate rows.

Part X — Verification & Acceptance A1 → A16

VERDICT A1 through A16 passed. The phase is not merely code-complete; it is route-tested, permission-tested, cache-tested, search-tested, pagination-tested, empty-state-tested, and latency-tested.

ID	Test	Expected	Final result
A1	Normal /dashboard/kpis	variant == my	PASS
A2	LIC /dashboard/kpis	variant == org	PASS
A3	Second SA dashboard call	cacheHit=true, cacheAgeMs>0	PASS (cacheAgeMs=199 ms)
A4	Write-time invalidate	JR + equipment code paths wired	PASS (code path)
A5	FE polling	30 second refresh behavior	PASS
A6	Manual refresh	User-triggered KPI reload	PASS
A7	Equipment utilization	Manual SQL ±1%	PASS (5704 active, open-work 0 → 0%)
A8	Vendor q=tech FULLTEXT	returns vendor hits	PASS (39 hits, first REACH TECHNOLOGIES)
A9	Normal /inquiry/job-cards	403 FORBIDDEN	PASS
A10	LIC /inquiry/job-cards	returns rows	PASS (19,432 total)
A11	q=ab (<3 chars)	prefix LIKE branch	PASS (2 hits)
A12	q=oscill	FULLTEXT BOOLEAN-prefix	PASS (484 hits)
A13	page=2	second slice of 10	PASS
A14	/inquiry?tab=products	opens Products tab directly	PASS
A15	no-match query	total=0, items=0 friendly state	PASS
A16	p50 latency	hit<5ms, miss<150ms, FT<50ms	PASS (3–4 ms hit, 3 ms miss, 6–30 ms FT)

Smoke Transcript Snapshot

A1	Normal /dashboard/kpis → variant==my	[my]
A2	LIC /dashboard/kpis → variant==org	[org]
A3	SA second call → cacheHit=true	[True 199]
A8	Vendor q=tech FULLTEXT	[total=39]
A9	Normal /inquiry/job-cards	[FORBIDDEN]
A10	LIC /inquiry/job-cards	[total=19432]
A11	q=ab prefix LIKE	[total=2]
A12	q=oscill FULLTEXT BOOLEAN-prefix	[total=484]
A13	page=2	[items=10]
A15	no-match	[total=0, items=0]
A16	cache-hit 3–4 ms · cache-miss 3 ms · inquiry FT 6–30 ms	

LESSON FROM A12 The only material implementation correction was the switch from Natural Language FULLTEXT to BOOLEAN-prefix search. This is exactly why acceptance tests matter: they protect user search intent, not just syntax.

Part XI — File Inventory & Sealed Manifest

Category	Net new	Patches	Deleted / replaced
DB migrations	3 idempotent scripts	none	none
Backend	11 new files	server.js, jobRequests.service.js, equipment.service.js	none
Frontend	11+ new phase files	App.jsx, permissions/nav	old Dashboard.jsx and InquiryPlaceholder.jsx replaced
Discovery / docs	SCHEMA_PHASE8.md	memory entry update	none
DDL changes	4 BTREE + 3 FULLTEXT + permission safety-net	0 ALTER column	0 DROP / 0 RENAME

Database Files

File	Purpose
120__phase8_kpi_indexes.sql	4 covering indexes for dashboard KPI aggregation.
121__phase8_fulltext_indexes.sql	FULLTEXT indexes: ft_cont_search, ft_prod_search, ft_eqip_search.
122__phase8_dashboard_inquiry_permissions.sql	INSERT IGNORE permissions and role grants safety-net.
runner/introspect_phase8.js	Read-only discovery script used to produce schema reality.
BE/db/discovery/SCHEMA_PHASE8.md	Canonical-to-legacy map and decision register.

Backend Files

Family	Files
Utility	src/utls/kpiCache.js
Dashboard	dashboard.validators.js, dashboard.repo.js, dashboard.service.js, dashboard.controller.js, dashboard.routes.js
Inquiry	inquiry.validators.js, inquiry.repo.js, inquiry.service.js, inquiry.controller.js, inquiry.routes.js
Patches	server.js route mounts; jobRequests.service.js and equipment.service.js invalidation hooks

Frontend Files

Family	Files
API	src/lib/api/dashboard.js, src/lib/api/inquiry.js
Hooks	src/lib/hooks/useDashboardKpis.js, src/lib/hooks/useInquirySearch.js
Schemas	src/lib/schemas/inquirySchemas.js
Dashboard pages	Dashboard.jsx, DashboardHeader.jsx, KpiGrid.jsx, KpiCard.jsx, QuickActions.jsx
Inquiry pages	Inquiry.jsx, InquiryTabs.jsx, SearchPanel.jsx, VendorResults.jsx, ProductResults.jsx, JobCardResults.jsx, InstrumentResults.jsx
Routing/nav	App.jsx route replacement and permissions.js navigation alignment

Part XII — How to Think Like the Phase 8 Engineer

Database Logic Development

MENTAL MODEL A dashboard number is not a UI label. It is a contract: entity scope + status definition + date boundary + index support + role visibility.

Question to ask	Example answer in Phase 8
Which entity owns the truth?	Calibration due belongs to cmms_equip_mst, not job cards.
What is the real status value?	Operational means EQM_MVP_STATUS = ACTIVE.
Which date boundary is intended?	Completed this week uses ISO week start.
Which role can see it?	ORG variant for LE/LIC/SA; MY variant for Normal/View-Only.
Which index supports it?	idx_equip_status_caldue and idx_jc_status_verified.

Backend Logic Development

MENTAL MODEL A backend endpoint is not just a route. It is a promise: authenticated identity, authorized permission, validated inputs, isolated SQL, business-shaped output, and predictable error behavior.

Layer	Good Phase 8 behavior	Bad behavior avoided
Route	authenticate → authorize → validate → controller	Skipping permission checks because UI hides links.
Controller	Pass actor/query to service	Writing SQL in controller.
Service	Branch by role, shape cards, cache payload	Returning raw DB rows to UI.
Repo	Alias legacy columns to canonical names	Leaking EQM_/JM_/JR_ columns everywhere.
Utility	Cache with clear keys and TTL	Global mutable magic without invalidation.

Frontend Logic Development

MENTAL MODEL Frontend pages are orchestration shells. They coordinate loading, filtering, empty states, and layout. They do not own the truth of permissions, roles, or KPI math.

Frontend rule	Phase 8 application
Use API wrappers	dashboard.js and inquiry.js keep HTTP details out of components.
Use hooks for async state	useDashboardKpis and useInquirySearch own loading/error/abort behavior.
Use URL for route state	/inquiry?tab=products is the tab source of truth.
Use permission gates for visibility	QuickActions filter by permission and routes still enforce backend gates.
Use friendly empty states	A15 proved no-match search is a first-class state.

Part XIII — Slice 2 Punchlist & Future Direction

Phase 8 Slice 1 is sealed. The following items are explicitly deferred and should not be re-debated unless DS opens a new phase/slice.

Deferred item	Why deferred	Likely next implementation
Dashboard charts	Cards shipped first; charts need more visual design and aggregate endpoints.	Workload bar, calibration timeline, equipment status pie.
Recent activity feed	Requires audited stream design and pagination.	GET /dashboard/activity reading audit_log.
SSE push	Polling is sufficient for Slice 1.	Server-sent events after cache invalidation strategy matures.
Inquiry row deep links	Vendor/product detail pages do not exist yet.	Add detail routes and row click only when pages are real.
SERVICE_PROVIDER vendor type	Legacy enum lacks it.	ADD enum value + backfill if business confirms.
Division calibration KPI	Scope requires division-user mapping and UX placement.	LE/LIC-specific card or filter in Slice 2.
Job Card lifecycle invalidation	Future JC mutations will affect dashboard counts.	Add hooks when JC mutation endpoints ship.

SEALED STATUS Phase 8 Slice 1 is SEALED · LOCKED · GREEN. It delivered the visibility layer: role-aware dashboard KPIs and four-tab Inquiry search over live legacy data, with additive migrations and verified performance.

Appendix A — Architecture Diagram



Appendix B — Status and Label Mapping

Raw DB value	UI / canonical meaning	Module
EQM_MVP_STATUS = ACTIVE	Operational / healthy equipment	Dashboard + Instrument Inquiry
EQM_MVP_STATUS = PENDING_VERIFICATION	Registered but awaiting verification	Instrument Inquiry
EQM_MVP_STATUS = UNDER_CALIBRATION	Currently in calibration	Instrument Inquiry
EQM_MVP_STATUS = UNDER_REPAIR	Currently in repair	Instrument Inquiry
JM_MVP_STATUS = ASSIGNED	Job card assigned	Job Card Inquiry
JM_MVP_STATUS = IN_PROGRESS	Work underway	Job Card Inquiry
JM_MVP_STATUS = COMPLETED	Completed but not necessarily verified	Dashboard + Job Card Inquiry
JM_MVP_STATUS = VERIFIED_CLOSED	Verified and closed	Dashboard + Job Card Inquiry
JR_MVP_STATUS = SUBMITTED	Pending approval / pending jobs	Dashboard

Appendix C — Final Seal Manifest

Seal item	Status
Phase 8 reads from live MySQL final	LOCKED

Seal item	Status
No mock data	LOCKED
No destructive DDL	LOCKED
Dashboard + Inquiry routes mounted	LOCKED
A1 → A16 smoke green	LOCKED
Performance inside target	LOCKED
Decision register P8-D1 → P8-D15	LOCKED
Submodules documented	LOCKED
Slice 2 punchlist explicit	LOCKED

Part XIV — Extended Step-by-Step Engineering Notebook

WHY THIS PART EXISTS This extended notebook expands every implementation step into a senior-engineer reasoning page: intent, design logic, files touched, risk avoided, and verification gate. This is the maximum-detail learning layer for DS and future maintainers.

STEP 0 — Read the sealed context and perform live introspection

Before writing a single route, Phase 8 validated the current repo and database state. This step protected us from implementing against imagined table names and ensured Phase 6/7 patterns remained the source of truth.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	FINAL-DESC, Phase 6/7 sealed docs, live MySQL final
Output	introspect_phase8.js and table reality matrix
Risk avoided	Building fake vendors/products/equipment tables
Verification	Existing tables, row counts, columns, and indexes printed
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 1 — Author SCHEMA_PHASE8.md

The schema document translated legacy database reality into canonical module language. It froze P8-D1 to P8-D15 and made implementation deterministic.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	Step 0 introspection
Output	Canonical-to-real mapping, decision register, gap list
Risk avoided	Leaking legacy column names into UI/services
Verification	Each dashboard/inquiry field has a real column or derived rule
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 2 — Resolve Q-1 through Q-9 and the 3 hidden schema questions

The Q&A step turned ambiguity into locked implementation choices. Product columns, vendor service-provider scope, and operational status mapping were resolved before code.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	Prompt requirements plus DB reality
Output	Green-flag decisions
Risk avoided	Mid-build schema redesign
Verification	DS accepted recommended answers
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 3 — Migration 120 KPI indexes

Dashboard cards are read-heavy aggregation paths. Migration 120 adds the indexes required for calibration due, personal calibration due, and completed-week calculations.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	KPI SQL paths
Output	idx_equip_status_caldue, idx_equip_creator_caldue, idx_jc_status_verified, idx_jc_status_ended
Risk avoided	Table scans on dashboard refresh
Verification	Indexes confirmed through information_schema
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 4 — Migration 121 FULLTEXT indexes

Inquiry must search quickly across vendor, product, and instrument text fields. Migration 121 creates FULLTEXT indexes without changing row structure.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	Inquiry search requirements
Output	ft_cont_search, ft_prod_search, ft Equip_search
Risk avoided	LIKE %term% scans on large equipment data
Verification	A8/A12 search tests green
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 5 — Migration 122 permission safety-net

The permission rows already existed from earlier seeds, but Phase 8 added a safety-net migration so fresh rebuilds are deterministic.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	RBAC matrix
Output	INSERT IGNORE permissions and grants
Risk avoided	Missing route permission on fresh DB
Verification	Permission grant counts matched matrix
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 6 — kpiCache utility

The cache prevents every dashboard refresh from recomputing all KPI aggregations. It mirrors Phase 7 tokenVersionCache style but uses dashboard-specific keys.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	P8-D2/D3
Output	LRU cache, 10s TTL, 5000 entries
Risk avoided	DB saturation during dashboard viewing
Verification	A3 cacheHit=true with cacheAgeMs>0
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 7 — Dashboard validators

The dashboard endpoint deliberately accepts no query parameters in Slice 1. The strict schema prevents invisible, unsupported filters from entering the API contract.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	Slice 1 scope
Output	kpisQuerySchema strict empty object
Risk avoided	Silent query drift
Verification	Route validation rejects unknown params
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 8 — Dashboard repository

The repo owns all KPI SQL. It isolates legacy tables and returns only canonical aggregate values to the service layer.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	SCHEMA_PHASE8 column map
Output	orgPendingJobs, orgCalibrationDue7d, orgCompletedThisWeek, orgEquipmentUtilization, personal KPI queries
Risk avoided	SQL scattered across service/controller
Verification	A1/A2/A7 data tests green
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 9 — Dashboard service

The service chooses ORG or MY variant, reads/writes kpiCache, formats card labels/subtitles, calculates utilization, and builds quick actions.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	req.user.role and employeeId
Output	cards, quick_actions, generatedAt, cache metadata
Risk avoided	Role logic duplicated in frontend
Verification	A1 my, A2 org, A3 cache hit
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 10 — Inquiry validators

Inquiry validators bound q length, page sizes, vendor type filter, and tab-specific future expansion. They are the first security boundary for search.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	P8-D11/P8-D12
Output	four query schemas
Risk avoided	unbounded fulltext strings and unsupported filters
Verification	A11, A12, A13, A15 green
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 11 — Inquiry repository

The Inquiry repo implements the four search paths and contains all legacy column knowledge. It also chooses prefix LIKE vs FULLTEXT BOOLEAN-prefix by q length.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	FULLTEXT indexes and alias map
Output	searchVendors, searchProducts, searchJobCards, searchInstruments
Risk avoided	SQL injection and slow unindexed search
Verification	A8, A10, A11, A12, A13 green
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 12 — Inquiry service

The service normalizes outputs into consistent pagination/results payloads, maps status labels, vendor types, and progress semantics.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	repo rows
Output	stable FE contract
Risk avoided	frontend-specific row transformation drift
Verification	UI renders every tab from the same response family
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 13 — server.js route mounts

Phase 8 added two module routers under /api/v1/dashboard and /api/v1/inquiry, preserving the existing middleware order.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	dashboard.routes.js and inquiry.routes.js
Output	live API endpoints
Risk avoided	unprotected ad-hoc endpoints
Verification	A1-A16 routes reachable
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 14 — Cache invalidation hooks

Read-side cache must be invalidated by write-side mutations. Job Request create/submit and Equipment create now clear the correct KPI keys after commit.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	P8-D15
Output	post-commit invalidation in services
Risk avoided	stale dashboard counts after a write
Verification	A4 code path wired
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 15 — Frontend API, hooks, pages, and routing

The frontend replaced placeholders with real pages. API wrappers stay thin, hooks own async state, pages own layout, and route permission gates remain in App.jsx.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	BE response contracts
Output	dashboard and inquiry UI modules
Risk avoided	axios calls scattered inside visual components
Verification	Vite build and browser/API smoke green
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

STEP 16 — Final smoke and sealing

The phase was sealed only after role, permission, search branch, pagination, empty state, cache, and latency checks passed.

SENIOR ENGINEERING INTENT This step exists to reduce one class of future failure. Phase 8 is designed as a chain of small, verifiable contracts rather than a large uncertain feature push.	
Dimension	Phase 8 answer
Input	Running BE+FE+DB
Output	A1 through A16 all green
Risk avoided	declaring done from code compile only
Verification	A16 timing within targets
LOCK After this step, the next layer can be built without guessing. That is the Phase 8 construction rhythm: prove reality, write contract, implement narrowly, verify quickly.	

Deep Dive — Dashboard KPI Card 1 — Pending Jobs

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Definition	Submitted job requests awaiting action
Primary table	cmms_jobrequest_mst
Filter	JR_MVP_STATUS = SUBMITTED
Why useful	Shows operational backlog before assignment
Failure mode	Using assigned/in-progress statuses would inflate backlog incorrectly
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Dashboard KPI Card 2 — Calibration Due

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Definition	Active equipment with calibration due date within 7 days
Primary table	cmms_equip_mst
Filter	EQM_MVP_STATUS = ACTIVE and EQM_CAL_DUE_DATE <= today + 7d
Index	idx_equip_status_caldue
Failure mode	Comparing to OPERATIONAL would return zero because OPERATIONAL does not exist
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Dashboard KPI Card 3 — Completed This Week

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Definition	Job cards completed or verified in current ISO week
Primary table	cmms_jobcard_mst
Date logic	COALESCE(JM_VERIFIED_ON, JM_JobEndDate)
Index	idx_jc_status_verified + idx_jc_status_ended
Failure mode	Rolling 7 days would not match management week reporting
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Dashboard KPI Card 4 — Equipment Utilization

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Definition	Active equipment currently tied to open job work
Primary tables	cmms equip_mst + cmms_jobcard_mst
Computation	open-work equipment / active equipment × 100
UI label	Optimal / below / above range
Failure mode	Counting all historical job cards would produce 100% utilization forever
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Inquiry Vendor Tab

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Source	cmms_cont_mst
Types	Manufacturer and Supplier only in Slice 1
Search fields	name, contact person, email
Index	ft_cont_search
Deferred	SERVICE_PROVIDER enum extension
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Inquiry Product Tab

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Source	cmms_product_mst
Reality	No manufacturer/category/supplier FK
Displayed	ID, name, description, equipment count, top manufacturer
Index	ft_prod_search
Decision	derive operational fields instead of ALTER
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Inquiry Job Card Tab

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Source	cmms_jobcard_mst + equipment join
Permission	Not available to Normal User
Search	job code prefix + equipment fulltext
Pagination	10/25 page size
Smoke	A10 returned 19,432 rows for LIC
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Inquiry Instrument Tab

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Source	cmms equip_mst + cmms_section_mst
Search	equipment name, model, serial, ID prefix
Index	ft_equip_search
Status mapping	ACTIVE shown as operational
Smoke	A12 q=oscill returned 484 hits
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — RBAC Deep Dive

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Dashboard	All five roles
Vendors/products/instruments	All five roles
Job cards inquiry	View-Only, Lab Engineer, Lab In-Charge, Super Admin
Normal user	403 on job-card inquiry
Why	Job cards are operational internals, not normal-user global lookup
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Performance Deep Dive

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Cache hit target	<5 ms
Cache miss target	<150 ms
Inquiry FULLTEXT target	<50 ms
Actual hit	3-4 ms
Actual FULLTEXT	6-30 ms
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Error Handling Deep Dive

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Validation errors	422 from zod pipeline
Forbidden route	403 from authorize middleware
Empty search	200 with total_items=0 and friendly UI
Unknown tab	client normalizes URL
Stale request	AbortController prevents old results flashing
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Deep Dive — Operational Handover Deep Dive

PURPOSE This page preserves the senior-engineering reasoning behind one high-leverage Phase 8 behavior.	
Point	Detail
Run order	DB migrations -> BE -> FE -> smoke
Rollback thinking	Indexes and INSERT IGNORE are additive; code rollback safe
Monitoring	Cache stats and request timings
Known followups	charts, activity, SSE, deep links
Seal rule	Do not expand Scope 1 after seal
REVIEW NOTE When debugging this feature later, start from this table: source table, status meaning, index support, permission gate, and smoke result.	

Final Seal — Phase 8 Fully Packed

FINAL VERDICT Phase 8 Dashboard + Inquiry Module WITH SUB-MODULES is END-TO-END COMPLETED. The phase is sealed as a read-heavy visibility layer over real CCMIS operational data.	
Axis	Sealed result
Database	3 additive migrations; 4 BTREE indexes; 3 FULLTEXT indexes; permission safety-net
Backend	Dashboard + Inquiry modules follow locked module template; SQL isolated in repos
Frontend	Dashboard and Inquiry placeholders replaced by real pages and hooks
Permissions	Route-level backend authorization and frontend navigation alignment
Performance	A16 confirms KPI and FULLTEXT latency inside targets
Documentation	This fully packed document records architecture, thinking, build steps, acceptance, and punchlist